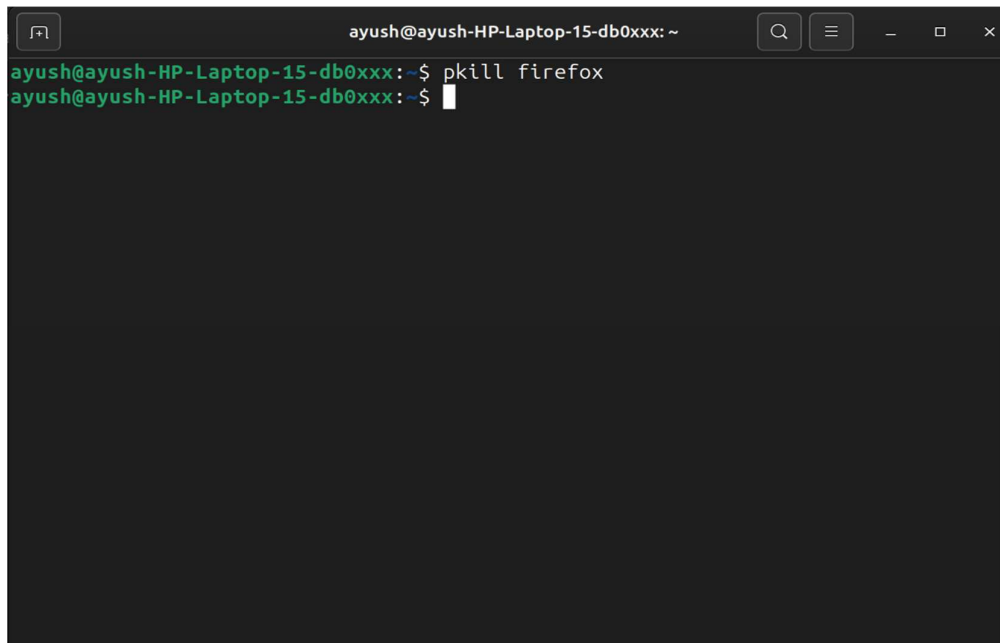


Practical-4 Output

1. Demonstrate different ways to kill processes in Linux.

a) Kill processes by name

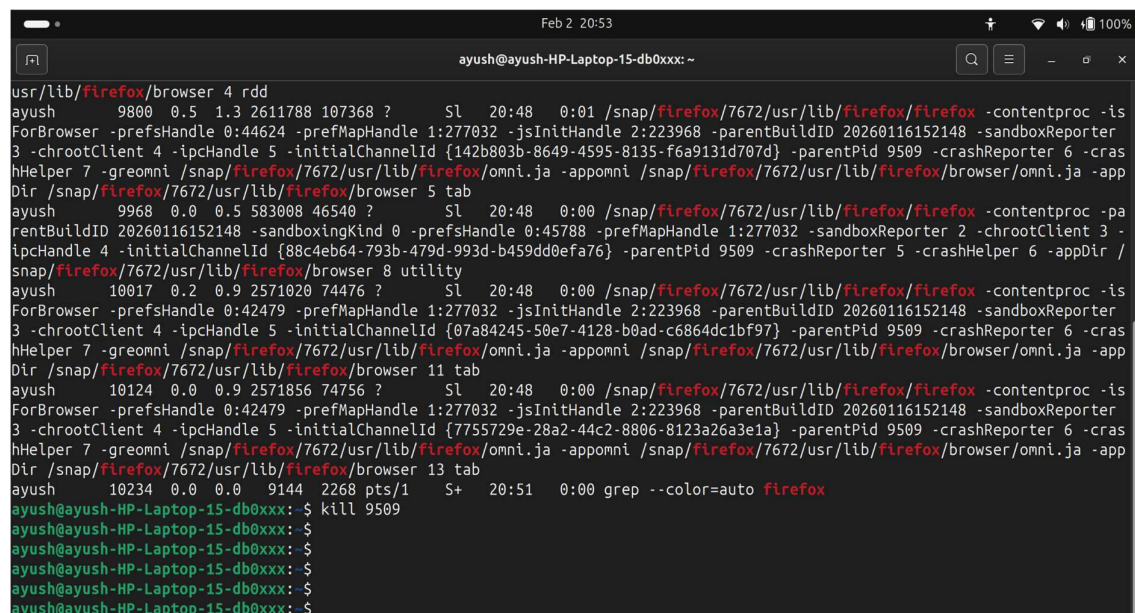
This method uses the `pkill` command to terminate all processes with a specific name.



```
ayush@ayush-HP-Laptop-15-db0xxx: ~  
ayush@ayush-HP-Laptop-15-db0xxx:~$ pkill firefox  
ayush@ayush-HP-Laptop-15-db0xxx:~$
```

b) Kill a process by finding its PID using process name

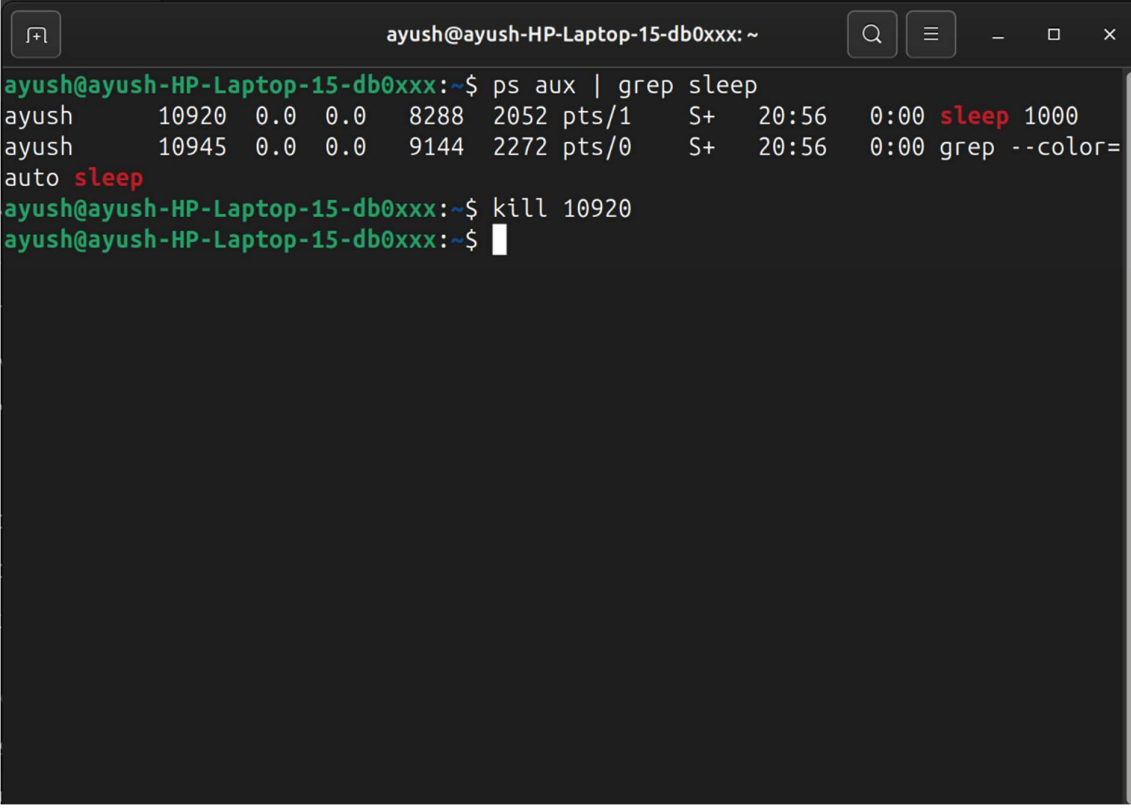
The `ps aux | grep` command is used to find the Process ID (PID), which is then terminated using the `kill` command.



```
usr/lib/firefox/browser 4 rdd  
ayush 9800 0.5 1.3 2611788 107368 ? Sl 20:48 0:01 /snap/firefox/7672/usr/lib/firefox/firefox -contentproc -is  
ForBrowser -prefsHandle 0:44624 -prefMapHandle 1:277032 -jsInitHandle 2:223968 -parentBuildID 20260116152148 -sandboxReporter  
3 -chrootClient 4 -ipcHandle 5 -initialChannelId {142b803b-8649-4595-8135-f6a9131d707d} -parentPid 9509 -crashReporter 6 -cras  
hHelper 7 -greomni /snap/firefox/7672/usr/lib/firefox/omni.ja -appomni /snap/firefox/7672/usr/lib/firefox/browser/omni.ja -app  
Dir /snap/firefox/7672/usr/lib/firefox/browser 5 tab  
ayush 9968 0.0 0.5 583008 46540 ? Sl 20:48 0:00 /snap/firefox/7672/usr/lib/firefox/firefox -contentproc -pa  
rentBuildID 20260116152148 -sandboxingKind 0 -prefsHandle 0:45788 -prefMapHandle 1:277032 -sandboxReporter 2 -chrootClient 3 -  
ipcHandle 4 -initialChannelId {88c4eb64-793b-479d-993d-b459dd0efa76} -parentPid 9509 -crashReporter 5 -crashHelper 6 -appDir /  
snap/firefox/7672/usr/lib/firefox/browser 8 utility  
ayush 10017 0.2 0.9 2571020 74476 ? Sl 20:48 0:00 /snap/firefox/7672/usr/lib/firefox/firefox -contentproc -is  
ForBrowser -prefsHandle 0:42479 -prefMapHandle 1:277032 -jsInitHandle 2:223968 -parentBuildID 20260116152148 -sandboxReporter  
3 -chrootClient 4 -ipcHandle 5 -initialChannelId {07a84245-50e7-4128-b0ad-c6864dc1bf97} -parentPid 9509 -crashReporter 6 -cras  
hHelper 7 -greomni /snap/firefox/7672/usr/lib/firefox/omni.ja -appomni /snap/firefox/7672/usr/lib/firefox/browser/omni.ja -app  
Dir /snap/firefox/7672/usr/lib/firefox/browser 11 tab  
ayush 10124 0.0 0.9 2571856 74756 ? Sl 20:48 0:00 /snap/firefox/7672/usr/lib/firefox/firefox -contentproc -is  
ForBrowser -prefsHandle 0:42479 -prefMapHandle 1:277032 -jsInitHandle 2:223968 -parentBuildID 20260116152148 -sandboxReporter  
3 -chrootClient 4 -ipcHandle 5 -initialChannelId {7755729e-28a2-44c2-8806-8123a26a3e1a} -parentPid 9509 -crashReporter 6 -cras  
hHelper 7 -greomni /snap/firefox/7672/usr/lib/firefox/omni.ja -appomni /snap/firefox/7672/usr/lib/firefox/browser/omni.ja -app  
Dir /snap/firefox/7672/usr/lib/firefox/browser 13 tab  
ayush 10234 0.0 0.0 9144 2268 pts/1 S+ 20:51 0:00 grep --color=auto firefox  
ayush@ayush-HP-Laptop-15-db0xxx:~$ kill 9509  
ayush@ayush-HP-Laptop-15-db0xxx:~$  
ayush@ayush-HP-Laptop-15-db0xxx:~$  
ayush@ayush-HP-Laptop-15-db0xxx:~$  
ayush@ayush-HP-Laptop-15-db0xxx:~$  
ayush@ayush-HP-Laptop-15-db0xxx:~$
```

c) Kill a single process using a given PID

A specific process can be terminated directly using its *PID* with the *kill* command.

A terminal window titled 'ayush@ayush-HP-Laptop-15-db0xxx: ~' with standard window controls. The terminal shows a user running 'ps aux | grep sleep', which lists two processes: 'sleep 1000' (PID 10920) and 'grep --color=' (PID 10945). The user then runs 'kill 10920', and the prompt returns without further output, indicating the process was successfully terminated.

```
ayush@ayush-HP-Laptop-15-db0xxx:~$ ps aux | grep sleep
ayush      10920  0.0  0.0  8288  2052 pts/1    S+   20:56   0:00  sleep 1000
ayush      10945  0.0  0.0  9144  2272 pts/0    S+   20:56   0:00  grep --color=
auto  sleep
ayush@ayush-HP-Laptop-15-db0xxx:~$ kill 10920
ayush@ayush-HP-Laptop-15-db0xxx:~$
```

2. Write C programs to demonstrate process states.

a) Orphan Process

An orphan process is created when a parent process terminates before its child, and the child is adopted by the init/systemd process.

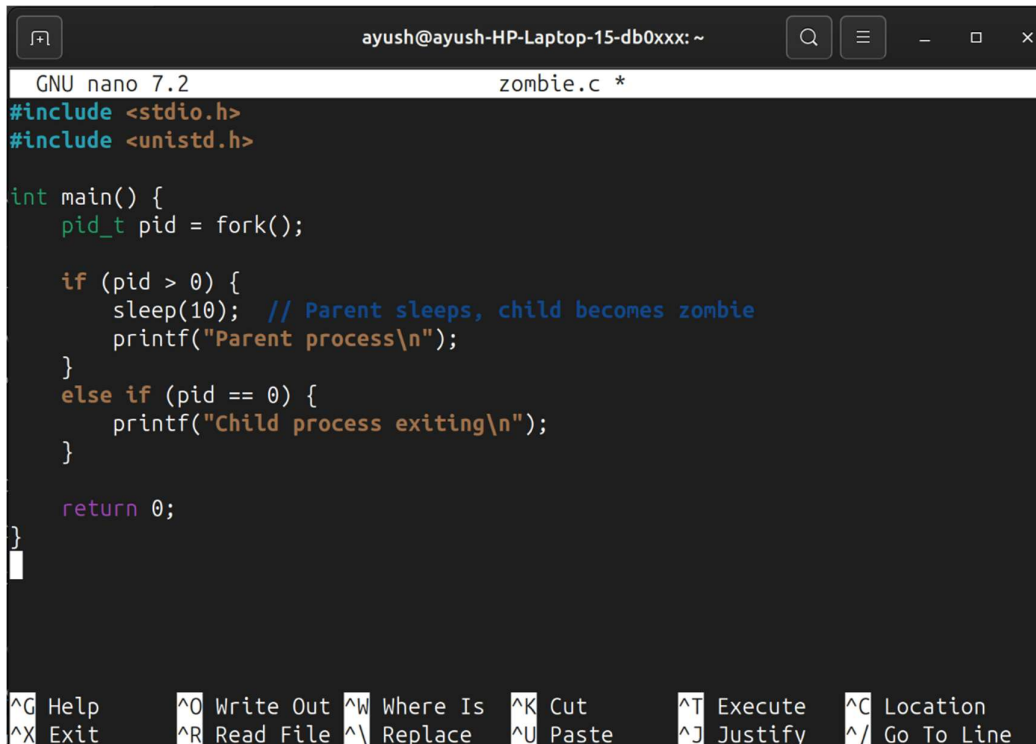
```
ayush@ayush-HP-Laptop-15-db0xxx: ~  
GNU nano 7.2 orphan.c *  
#include <stdio.h>  
#include <unistd.h>  
  
int main() {  
    pid_t pid = fork();  
  
    if (pid > 0) {  
        printf("Parent process exiting...\n");  
    }  
    else if (pid == 0) {  
        sleep(5);  
        printf("Child process running...\n");  
        printf("New Parent PID: %d\n", getppid());  
    }  
  
    return 0;  
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line

```
ayush@ayush-HP-Laptop-15-db0xxx: ~  
ayush@ayush-HP-Laptop-15-db0xxx:~$ nano orphan.c  
ayush@ayush-HP-Laptop-15-db0xxx:~$ gcc orphan.c -o orphan  
ayush@ayush-HP-Laptop-15-db0xxx:~$ ./orphan  
Parent process exiting...  
ayush@ayush-HP-Laptop-15-db0xxx:~$ Child process running...  
New Parent PID: 1270
```

b) Zombie Process

A zombie process occurs when a child process completes execution but still remains in the process table because the parent has not collected its exit status.



```
GNU nano 7.2 zombie.c *
#include <stdio.h>
#include <unistd.h>

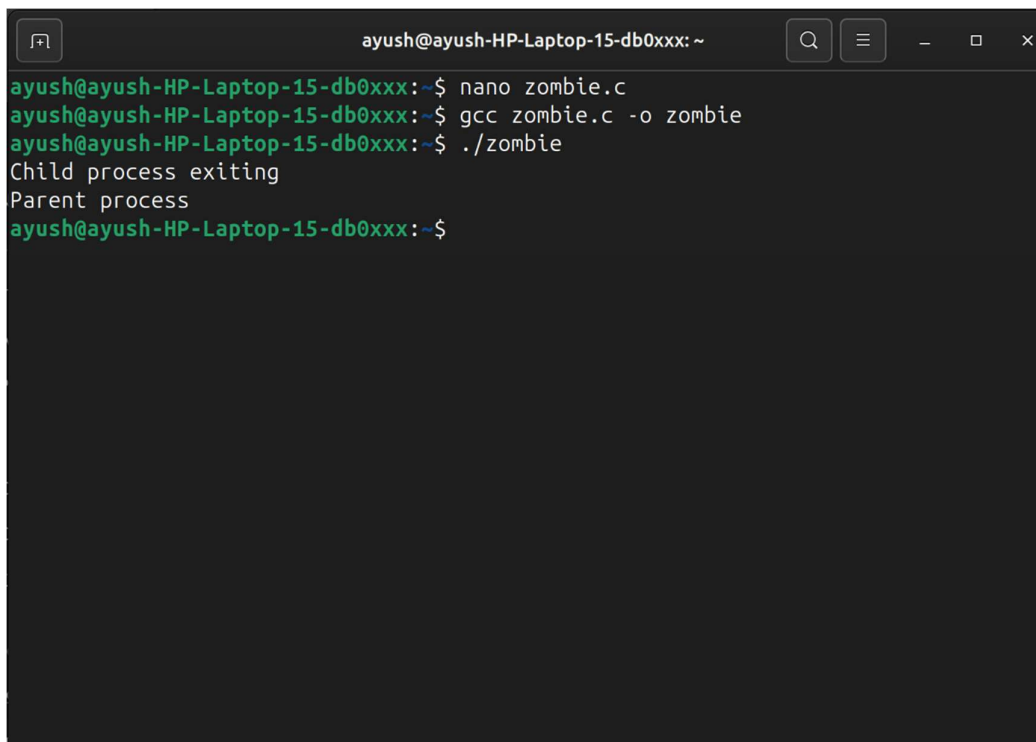
int main() {
    pid_t pid = fork();

    if (pid > 0) {
        sleep(10); // Parent sleeps, child becomes zombie
        printf("Parent process\n");
    }
    else if (pid == 0) {
        printf("Child process exiting\n");
    }

    return 0;
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line

Terminal-1:



```
ayush@ayush-HP-Laptop-15-db0xxx:~$ nano zombie.c
ayush@ayush-HP-Laptop-15-db0xxx:~$ gcc zombie.c -o zombie
ayush@ayush-HP-Laptop-15-db0xxx:~$ ./zombie
Child process exiting
Parent process
ayush@ayush-HP-Laptop-15-db0xxx:~$
```

Terminal-2:

```
ayush@ayush-HP-Laptop-15-db0xxx: ~  
ayush@ayush-HP-Laptop-15-db0xxx:~$ ps aux | grep defunct  
ayush      9283  0.0  0.0      0   0 ?        Z    20:46   0:00 [sd_espeak-ng  
-mb] <defunct>  
ayush     14103  0.0  0.0   9144 2272 pts/1    S+   21:07   0:00 grep --color=  
auto defunct  
ayush@ayush-HP-Laptop-15-db0xxx:~$
```

3. Write a C program to create a child process using fork() and display PID and PPID.

This program demonstrates process creation using fork() and shows the Process ID (PID) and Parent Process ID (PPID) of both parent and child processes.

```
GNU nano 7.2                                fork.c *
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
    }
    else if (pid == 0) {
        printf("This is Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID (PPID): %d\n", getppid());
    }
    else {
        printf("This is Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
    }

    return 0;
}
```

Help Write Out Where Is Cut Execute Location M-U Undo M-A Set Mark
Exit Read File Replace Paste Justify Go To Line M-E Redo M-6 Copy

```
ayush@ayush-HP-Laptop-15-db0xxx: ~
ayush@ayush-HP-Laptop-15-db0xxx:~$ nano fork.c
ayush@ayush-HP-Laptop-15-db0xxx:~$ gcc fork.c -o fork
ayush@ayush-HP-Laptop-15-db0xxx:~$ ./fork
This is Parent Process
Parent PID: 15504
Child PID: 15505
This is Child Process
Child PID: 15505
Parent PID (PPID): 15504
ayush@ayush-HP-Laptop-15-db0xxx:~$
```